

AIFP Quick Start Guide

Document: AIFP-DOC-07 · **Reading time:** ~5 minutes · **Governed by:** AIFP-1 (Doc 01)

Get paid (merchant) or pay autonomously (agent) with the ****AiFinPay Paywall**

Protocol (AIFP)** in minutes. AIFP turns HTTP `402 Payment Required` into a real, machine-payable response so AI agents can buy API calls, data, and compute without a human in the loop.

The 4-step loop

`402 challenge` → `POST /v1/quote` → `POST /v1/pay` (get receipt) → retry with `X-AIFP-Receipt`.

0. Prerequisites (30 seconds)

You need	Where
Base URL	Sandbox <code>https://sandbox.api.aifinpay.io</code> · Prod <code>https://api.aifinpay.io</code>
(Agents) a funded test wallet	created in Agent Quick Start below

Pricing is action-tier based:

Tier	Starts From	Typical action
Complex	\$0.00006	Search, aggregation, multi-source queries, higher compute
Premium	\$0.00010	AI inference, GPU workloads, deep analytics, premium data

AiFinPay charges a **1% protocol fee** on every successful transaction. The remaining **99%** is settled to the merchant, excluding any applicable payment network or settlement costs.

1. Merchant Quick Start (paywall your API)

Goal: return a `402` AIFP challenge when an agent must pay for an action, then verify the receipt it brings back.

a. Install

```
npm install @aifinpay/merchant # Node
```

```
pip install aifinpay-merchant # Python
```

b. Wrap your endpoint (Express)

```
import express from "express";
import { aifp } from "@aifinpay/merchant";

const app = express();

// Charges the Standard action tier (from $0.000001) for /api/data.
app.use("/api/data", aifp.protect({
  apiKey: process.env.AIFP_KEY, // sk test ..
  merchantId: "mrch_9f3a1c2b",
  pricingTier: "standard"
}));

app.get("/api/data", (req, res) => res.json({ data: "premium payload" }));
app.listen(3000);
```

The middleware does everything in AIFP-1 §7.4: if no valid receipt, it returns a

402 challenge; if a receipt is present it verifies the Ed25519 signature **locally**

(no network call), checks `aud`, `resource`, `amount`, `exp`, and `nonce`, then calls your handler.

c. Verify a receipt manually (any language)

```
from aifinpay_merchant import verify_receipt
ok, claims = verify_receipt(jwt, merchant_id="mrch_9f3a1c2b", resource="/api/data")
ok == True only if signature + aud + resource + amount + exp + unused-nonce all pass
```

You're live. Settlement (stablecoin or hybrid fiat via BVNK) lands at your payout address after the 1% AiFinPay protocol fee and any network or settlement costs.

2. Agent Quick Start (pay for a resource)

Goal: an autonomous agent hits a paywalled API and pays itself through.

a. Install

```
npm install @aifinpay/agent # Node / TS
pip install aifinpay-agent # Python
```

b. Pay-through in one call

```
from aifinpay_agent import Agent

agent = Agent(api_key="sk_test...", wallet_id="wlt_3a1b")
```

```
# The SDK runs the full loop automatically:
# 402 -> /v1/quote -> /v1/pay -> retry with X-AIFP-Receipt
resp = agent.get("https://merchant.example.com/api/data")
print(resp.json())           # { "data": "premium payload" }
print(resp.aifp_receipt_id) # rcpt 7b3e9f21
```

c. What happened under the hood

```
GET /api/data -> 402 + AIFP challenge (qt uri, nonce, from $0.00001)
POST /v1/quote {merchant,resource} -> 200 quote_id=qt_8d21f6
POST /v1/pay {quote_id,wallet,asset} -> 200 receipt (EdDSA JWT), tx ref=0xabc...
GET /api/data X-AIFP-Receipt: <jwt> -> 200 payload
```

Set a spend cap so the agent can't overspend (returns AIFP-403-BUDGET-EXCEEDED):

```
agent.set_budget(window="day", cap_usd="50.00")
```

3. Wallet Quick Start

```
from aifinpay.agent import Wallet

# Non-custodial wallet (keys stay with you), custodial also supported
wallet = Wallet.create(api_key="sk_test", agent_id="agt_4f9a2c7e", type="non_custodial")
print(wallet.wallet_id)           # wlt_3a1k

# Fund it (sandbox faucet)
wallet.fund_test(asset="USDC", amount="25.00", chain="polygon")
print(wallet.balances())          # {"USDC": "25.00"}
```

Supported assets: **USDC, USDT, PYUSD**. Supported chains: 12 networks — Full Core

(8) Solana, Polygon, Avalanche, BNB Chain, Optimism, Arbitrum, Base, Unichain;

Splitter-only EVM (2) BOT Chain, XRPL EVM; MVP non-EVM (2) NEAR, Aptos.

4. SDK Installation Matrix

Language	Agent SDK	Merchant SDK
Python	<code>pip install aifinpay-agent</code>	<code>pip install aifinpay-merchant</code>
Go	<code>go get github.com/aifinpay/aifp-go</code>	same module
Rust	<code>cargo add aifinpay</code>	same crate
Java	<code>implementation "io.aifinpay:aifp:1.0.0"</code>	same
PHP	<code>composer require aifinpay/aifp</code>	same

Language	Agent SDK	Merchant SDK
C#	<code>dotnet add package AiFinPay</code>	same

Full method reference: **Doc 11 — SDK Reference**.

5. Running Your First Demo

```
# Clone the official examples
git clone https://github.com/aifinpay/examples
cd examples/quickstart

# Terminal 1 - start a paywalled merchant on :3000
AIFP KEY=sk_test ... node merchant.js

# Terminal 2 - run an agent that pays through
AIFP KEY=sk_test ... node agent.js

# -> 402 detected, quoted Standard from $0.00001, paid, receipt rcpt_..., got payload
```

6. Testing

```
# Dry-run the full loop without spending (sandbox)
aifp demo --sandbox --resource /api/data --tier standard

# Assert receipt verification
aifp verify --receipt "$JWI" --merchant mrch_9f3a1c2b --resource /api/data
```

Use the **Postman collection (Doc 09)** for click-through testing — it auto-chains

quote → pay → receipt and auto-generates the `Idempotency-Key` .

7. Sandbox vs Production

	Sandbox	Production
Keys	<code>sk_test_...</code>	<code>sk_live_...</code>
Settlement	simulated, faucet funds	real stablecoin / hybrid fiat
JWKS	test <code>kid</code>	rotating prod <code>kid</code>

To go live: swap the base URL + key, point your payout address to a real wallet, and

re-run the demo. No code changes.

8. Troubleshooting

Symptom	Cause	Fix
AIFP-422-SIGNATURE	Wrong JWKS / stale kid	Refresh /.well-known/jwks.json ; check kid
AIFP-403-BUDGET-EXCEEDED	Spend cap hit	Raise budget or wait for window reset
AIFP-409	Reused nonce / idempotency conflict	Get a fresh quote; new Idempotency-Key
AIFP-410	Quote expired	Re-quote (quotes are short-lived)
AIFP-425	Settlement not confirmed	Honor Retry-After , retry same receipt_id
AIFP-429	Rate limited	Exponential backoff; honor Retry-After

Still stuck? See the full error registry in **AIFP-1 Appendix C**, or open an issue at

github.com/aifinpay/aifp (Doc 15).

Next: Doc 02 (Merchant Integration Guide, 15 frameworks) · Doc 03 (Agent SDK Spec) ·

Doc 11 (SDK Reference) · Doc 08 (OpenAPI).